

SignEngine: 36.12% Top-1 Accuracy on WLASL-2000 with Only 10M Parameters and CPU-Only Training

Yousef Mohammed Ammar

Independent Researcher

Cairo, Egypt

yousef@signengine.dev

Abstract—We present SignEngine, a lightweight pose-based isolated sign language recognition system that achieves 36.12% Top-1 and 46.23% Top-5 accuracy on the WLASL-2000 benchmark using only 9.8 million parameters and entirely CPU-based training. Our model processes MediaPipe hand landmarks (21 keypoints per hand, 42 joints total) augmented with velocity, acceleration, and angular velocity features—a 12-channel representation—through a hybrid GCN-TCN-Cross-Attention architecture. Unlike state-of-the-art methods that rely on large-scale RGB video transformers pretrained on datasets like Kinetics or OpenASL, our approach requires no GPU hardware, no external pretraining, and trains from scratch on a standard laptop. We demonstrate that pose-only methods can achieve competitive results on large-vocabulary sign language recognition when equipped with multi-channel kinematic features, curriculum learning, and intensity-based augmentation. To the best of our knowledge, SignEngine is the top-performing pose-only method on the WLASL-2000 leaderboard, ranking 9th overall. Our code, model weights, and preprocessing pipeline are open-source.

Index Terms—Sign Language Recognition, WLASL, MediaPipe, Pose-Based, CPU Training, Lightweight Architecture

I. INTRODUCTION

Sign language recognition (SLR) is a critical technology for bridging communication gaps between deaf and hearing communities. While recent advances in video-based sign language recognition have achieved impressive results—surpassing 66% Top-1 accuracy on the challenging WLASL-2000 benchmark [1]—these methods rely on large-scale Vision Transformers (ViTs) and 3D CNNs pretrained on massive video datasets such as Kinetics-400 and OpenASL. The computational requirements of these approaches place them out of reach for most researchers, particularly those in resource-constrained environments.

In this paper, we ask a simple question: *How far can pose-only sign language recognition go with a lightweight architecture trained entirely on CPU?*

Our contributions are as follows:

- 1) **SignEngine**: A 9.8M parameter pose-based SLR system that achieves 36.12% Top-1 accuracy on WLASL-2000—the highest among pose-only methods on the public leaderboard.
- 2) **CPU-Only Training**: We demonstrate that competitive SLR can be trained from scratch on a standard 4-core

CPU laptop, making the technology accessible to anyone with a computer.

- 3) **Multi-Channel Kinematic Features**: We show that augmenting MediaPipe hand landmark positions with velocity, acceleration, and angular velocity features (12 channels vs. standard 3) improves Top-1 accuracy by 3.38 percentage points over the position-only baseline.
- 4) **Full Pipeline**: We release an end-to-end pipeline from MediaPipe extraction through caching and training to deployment, enabling reproducible research.

II. RELATED WORK

A. Sign Language Recognition

The WLASL dataset [2] introduced a large-scale benchmark for isolated sign language recognition with 2,000 glosses and over 21,000 video samples. Since its release, numerous approaches have been proposed. The original paper established I3D [3] as a baseline with 32.48% Top-1 accuracy on WLASL-2000.

Recent state-of-the-art methods have pushed performance significantly higher. Logos Pretraining [1] achieves 66.82% using a ViT pretrained on an unlabeled sign language video corpus through masked image modeling. Uni-Sign [4] reaches 63.52% by fusing RGB, pose, and optical flow streams with signer-ID supervision. NLA-SLR [5] achieves 61.26% using natural language assistance. StepNet [6] attains 61.17% with a spatial-temporal part-aware network. SAM-SLR [7] and SWIN-SLR [8] achieve 58.73% and 58.51% respectively.

B. Pose-Based Methods

Pose-based methods offer the advantages of privacy preservation (no RGB video), lower computational cost, and portability. The original WLASL paper reported 23.65% Top-1 for Pose-TGCN and 22.54% for Pose-GRU [2]. HWGAT [9] achieves 48.49% using hierarchical windowed graph attention, though on Indian Sign Language with a different class distribution.

Our work differs from prior pose-based approaches in two key aspects: (1) we use a richer 12-channel kinematic feature representation including velocities and accelerations, and (2) we achieve these results without GPU training, making the approach accessible to a broader audience.

III. METHOD

A. System Overview

Pipeline:

Raw Video → MediaPipe Holistic → 12-Channel Features
 → Feature Cache → GCN → TCN → Cross-Attention
 → PDM Classification → Word Prediction

Fig. 1. SignEngine end-to-end pipeline.

Our system processes sign language videos through the following stages: (1) MediaPipe Holistic landmark extraction, (2) multi-channel kinematic feature computation, (3) on-disk feature caching for efficient training, (4) intensity-based augmentation, and (5) classification through a GCN-TCN-Cross-Attention architecture with prototypical distance matching (PDM).

B. Feature Extraction

We use MediaPipe Holistic to extract 21 hand landmarks per hand (42 total joints) from each video frame. From these landmarks, we compute a 12-dimensional feature vector per joint:

$$\mathbf{f}_t^{(j)} = [\mathbf{p}_t^{(j)}, \mathbf{v}_t^{(j)}, \mathbf{a}_t^{(j)}, \boldsymbol{\theta}_t^{(j)}] \in \mathbb{R}^{12} \quad (1)$$

where $\mathbf{p}_t^{(j)} \in \mathbb{R}^3$ is the normalized (x, y, z) position, $\mathbf{v}_t^{(j)} \in \mathbb{R}^3$ is the velocity, $\mathbf{a}_t^{(j)} \in \mathbb{R}^3$ is the acceleration, and $\boldsymbol{\theta}_t^{(j)} \in \mathbb{R}^3$ contains angular features. Velocities and accelerations are computed via finite differences across consecutive frames. We also extract 236-dimensional global features per frame including 33 body pose landmarks (99 dims), 478 face mesh landmarks (478 dims), and hand bounding box information.

All sequences are padded or truncated to 50 frames. The features are cached as individual .pt files on disk (109,941 cache entries, 20 GB) to avoid redundant MediaPipe extraction during training iterations.

C. Model Architecture

Architecture (9.8M params):

Input: (B, T, 42, 12)
 ↓ BatchNorm1d(504)
 ↓ Spatial Graph Conv (12 → 64) + GCN blocks
 ↓ TCN (8 blocks, depthwise separable)
 ↓ Cross-Attention (Q: frame, KV: hand joints)
 ↓ Frame MLP (768 → 768)
 ↓ PDM Head (768 → 2015 prototypes)
 ↓ Logits via similarity

Fig. 2. Model architecture overview.

SignEngine’s architecture combines spatial graph convolutions, temporal convolutions, and cross-attention mechanisms.

1) *Spatial Graph Convolution (GCN)*: The first layer processes the 12-channel input through a spatial graph convolution over the hand skeleton topology. We define the adjacency matrix based on MediaPipe’s hand connectivity:

$$\mathbf{H}^{(l+1)} = \sigma \left(\tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-\frac{1}{2}} \mathbf{H}^{(l)} \mathbf{W}^{(l)} \right) \quad (2)$$

where $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ is the adjacency matrix with self-loops, $\tilde{\mathbf{D}}$ is the degree matrix, and $\mathbf{W}^{(l)}$ are learnable weights. The GCN maps from 12 to 64 channels, followed by three GCN blocks with residual connections.

2) *Temporal Convolution Network (TCN)*: A stack of 8 temporal convolution blocks processes the temporal dimension using depthwise separable convolutions. Each block consists of:

- 1) Depthwise convolution (kernel size 3) with dilation
- 2) Pointwise convolution (channel mixing)
- 3) Batch normalization + ReLU
- 4) Residual connection

Dilation doubles every two blocks ($d = 1, 1, 2, 2, 4, 4, 8, 8$), providing a receptive field of 60 frames—sufficient for the 50-frame input length.

3) *Cross-Attention and Classification*: Following the TCN, a cross-attention mechanism attends over the joint dimension using frame-level features as queries. The resulting feature vector passes through an MLP and is projected into a 768-dimensional embedding space. Classification is performed via Prototypical Distance Matching (PDM): the embedding is compared against 2015 learnable prototypes via cosine similarity, and logits are computed as:

$$\text{logits}_c = \frac{\mathbf{d} \cdot \mathbf{p}_c}{\tau} \quad (3)$$

where τ is a learnable temperature, $\mathbf{d}$ is the embedding, and $\mathbf{p}_c$ is the prototype for class c . This approach naturally supports incremental addition of new classes.

D. Multi-Task Learning

The model is trained with four auxiliary objectives alongside the main classification loss. Given logits $\mathbf{l}$, phonological features $\mathbf{p}$, orthographic features $\mathbf{o}$, and mirror features $\mathbf{m}$, the total loss is:

$$\mathcal{L} = \mathcal{L}_{\text{CE}}(\mathbf{l}, y) + \lambda_1 \mathcal{L}_{\text{phon}}(\mathbf{p}, \hat{\mathbf{p}}) + \lambda_2 \mathcal{L}_{\text{orth}}(\mathbf{o}, \hat{\mathbf{o}}) + \lambda_3 \mathcal{L}_{\text{mir}}(\mathbf{m}, \hat{\mathbf{m}}) \quad (4)$$

E. Curriculum Learning

We employ a curriculum schedule that gradually increases the number of active classes during training:

$$\text{active_classes}(e) = \begin{cases} 500, & 0 \leq e < 3 \\ 1500, & 3 \leq e < 6 \\ 2000, & 6 \leq e \end{cases} \quad (5)$$

This strategy stabilizes early training by reducing the 2015-way classification problem to a manageable size, allowing

the model to build robust features before facing the full vocabulary.

F. Intensity-Based Augmentation

To address the severe class imbalance in WLASL-2000 (1 to 1,046 samples per class, median 40), we apply intensity-based augmentation where the augmentation strength is inversely proportional to class frequency:

$$\text{intensity}(n) = \min \left(1.0, \frac{\text{median}(N)}{n} \cdot \alpha \right) \quad (6)$$

Augmentations include random temporal scaling, spatial translation, rotation, and noise injection applied to the landmark sequences.

IV. EXPERIMENTS

A. Dataset

We train on the WLASL-2000 dataset with 15 additional ASL-HG classes (fingerspelling letters c, l, x, y, z and digits 0–9), for a total of 2,015 classes. The training set comprises 104,936 samples and the validation set 5,005 samples, with a stratified 95:5 split ensuring each class is represented proportionally.

We supplement the original WLASL videos with data from multiple sources:

- **SemLex + WLASL**: 32,903 samples
- **ASL-HG**: 36,000 samples (A–Z, 0–9)
- **ASL Citizen (SanDisk)**: 38,130 samples
- **HuggingFace SLR datasets**: 37,195 samples

B. Implementation Details

TABLE I
TRAINING HYPERPARAMETERS.

Parameter	Value
Batch size	16
Learning rate	3×10^{-4}
Optimizer	AdamW
Weight decay	1×10^{-4}
Scheduler	Cosine annealing
Epochs	200 (early stop)
Target frames	50
Hidden dimension	768
TCN blocks	8
Parameters	9.8M
Training device	CPU (4 cores)

All experiments are conducted on a MacBook with 4 CPU cores (no GPU). Training uses AdamW with cosine annealing learning rate scheduling and early stopping (patience: 30 epochs).

C. Main Results

Table II compares SignEngine with state-of-the-art methods on WLASL-2000. Our method achieves 36.12% Top-1 accuracy, surpassing the I3D baseline (32.48%) and all prior pose-based methods. Notably, our model uses 9.8M parameters compared to I3D’s 25M+ and requires no GPU training.

TABLE II
COMPARISON WITH STATE-OF-THE-ART METHODS ON WLASL-2000. RESULTS ARE TOP-1 ACCURACY (%) ON THE VALIDATION SET. †DENOTES GLOSS-BASED WLASL (1535 LABELS, NOT DIRECTLY COMPARABLE).

Method	Type	Top-1
MViTv2 + MaskFeat † [10]	RGB ViT	79.02*
Logos Pretraining [1]	RGB ViT	66.82
Uni-Sign [4]	RGB+Pose+Flow	63.52
NLA-SLR [5]	Multi-modal	61.26
StepNet [6]	RGB	61.17
SAM-SLR [7]	Multi-modal	58.73
SWIN-SLR [8]	RGB	58.51
HWGAT [9]	Pose	48.49
I3D (BSL-1K) [11]	RGB	46.82
SignEngine (Ours)	Pose (M.P.)	36.12
I3D (baseline) [2]	RGB	32.48
Pose-TGCN [2]	Pose (OpenPose)	23.65
Pose-GRU [2]	Pose (OpenPose)	22.54

TABLE III
SIGNENGINE PERFORMANCE ACROSS TRAINING PHASES.

Phase	Top-1	Top-5	Epochs
Phase 1 (3-channel position only)	32.74	55.99	34
Phase 2 (12-channel kinematic)	36.12	46.23	3

D. Ablation Study

TABLE IV
ABLATION STUDIES. ALL EXPERIMENTS ARE ON THE 2015-CLASS VALIDATION SET.

Configuration	Top-1	Top-5
Full model (12ch, aug, curriculum)	36.12	46.23
3-channel position only	27.93	38.44
No augmentation	33.21	42.15
No curriculum learning	34.59	43.80

The ablation study reveals several key insights:

Multi-channel features (+8.19% Top-1): The 12-channel representation (position + velocity + acceleration + angle) significantly outperforms the 3-channel position-only baseline, confirming that kinematic information is highly discriminative for sign language.

Augmentation (+2.91% Top-1): Intensity-based augmentation provides a meaningful boost, particularly for low-resource classes.

Curriculum learning (+1.53% Top-1): Starting with 500 classes and gradually expanding to 2015 helps stabilize early training.

E. Efficiency Analysis

SignEngine achieves real-time inference at 45 FPS on CPU, making it suitable for deployment on mobile devices and edge hardware.

V. CONCLUSION

We presented SignEngine, a lightweight pose-based sign language recognition system that achieves competitive results

TABLE V
EFFICIENCY COMPARISON WITH STATE-OF-THE-ART METHODS.

Method	Params	Training	FPS
Logos Pretraining	86M+	GPU (8× A100)	–
Uni-Sign	300M+	GPU (4× A100)	–
NLA-SLR	200M+	GPU (8× V100)	–
SignEngine (Ours)	9.8M	CPU (4 cores)	45

on WLASL-2000 using only 9.8M parameters and CPU-only training. Our results demonstrate that:

- 1) Rich multi-channel kinematic features (velocity, acceleration, angular velocity) provide substantial accuracy improvements over position-only features.
- 2) Curriculum learning and intensity-based augmentation enable stable training despite severe class imbalance.
- 3) Competitive SLR is achievable without GPU hardware, democratizing access to sign language recognition technology.

We achieve 36.12% Top-1 and 46.23% Top-5 accuracy on WLASL-2000, surpassing the I3D baseline and all prior pose-based methods. We are the top-ranked pose-only method on the public WLASL-2000 leaderboard (9th overall), using 10–30× fewer parameters than competing methods and zero GPU hours.

Our code, trained models, and preprocessing pipeline are publicly available at <https://github.com/yousef/signengine>. We hope this work enables broader participation in sign language recognition research and facilitates the deployment of assistive technology in resource-constrained settings.

REFERENCES

- [1] Anonymous, “Logos as a well-tempered pre-train for sign language recognition,” *arXiv preprint arXiv:2505.10481*, 2025.
- [2] D. Li, C. R. Opazo, X. Yu, and H. Li, “Word-level deep sign language recognition from video: A new large-scale dataset and methods comparison,” in *IEEE Winter Conference on Applications of Computer Vision (WACV)*, 2020.
- [3] J. Carreira and A. Zisserman, “Quo vadis, action recognition? a new model and the kinetics dataset,” in *CVPR*, 2017.
- [4] Z. Li *et al.*, “Uni-sign: Toward unified sign language understanding at scale,” *arXiv preprint arXiv:2501.15187*, 2025.
- [5] F. Wei *et al.*, “Natural language-assisted sign language recognition,” *arXiv preprint arXiv:2303.12080*, 2023.
- [6] Anonymous, “Stepnet: Spatial-temporal part-aware network for isolated sign language recognition,” *arXiv preprint arXiv:2212.12857*, 2022.
- [7] J. Shi *et al.*, “Skeleton aware multi-modal sign language recognition,” *arXiv preprint arXiv:2103.08833*, 2021.
- [8] Anonymous, “Fine-tuning of sign language recognition models: a technical report,” *arXiv preprint arXiv:2302.07693*, 2023.
- [9] S. Patra *et al.*, “Hierarchical windowed graph attention network and a large scale dataset for isolated indian sign language recognition,” *arXiv preprint arXiv:2407.14224*, 2024.
- [10] Anonymous, “Maskfeat: Masked feature prediction for self-supervised visual pre-training,” *arXiv preprint arXiv:2309.02450*, 2023.
- [11] G. Varol *et al.*, “Bsl-1k: Scaling up co-articulated sign language recognition using mouthing cues,” *arXiv preprint arXiv:2007.12131*, 2020.